NEXUS-RESOLVE

Complete User Guide

Setup, Roles, Settings, Demo Operation, Audit Export, E2E Proof, and Pre/Post Dos

Project: Policy-grounded AI remediation for synthetic enterprise infrastructure operations

Workspace: D:\Hackathon - Copy\nexus-resolve

Reference commit: 4f87cd5 Add audit exports telemetry and E2E proof

Generated: 2026-06-27 16:23

This guide is written for judges, presenters, operators, approvers, developers, QA reviewers, and maintainers. It explains exactly how to run, use, verify, and safely present the project without exposing secrets or implying real host remediation.

Safety boundary: the demo is synthetic and mock-only. Do not claim that GitHub Pages proves live OpenAI usage, and do not put an OpenAI API key in frontend code or public GitHub.

Contents

1. Executive Summary

NEXUS-RESOLVE is a hackathon-grade operations console for safe, policy-grounded remediation. It takes synthetic enterprise infrastructure alerts, retrieves SOP evidence, compares historical tickets, blocks unsafe precedent, generates an approval-gated mock remediation plan, validates synthetic metrics, and produces RCA plus audit evidence.

The project has two main operating modes: public replay mode for safe static hosting, and local live mode for backend-backed WebSocket events and optional OpenAI Responses API structured outputs. The public GitHub Pages experience is intentionally replay-only and does not contain secrets.

The strongest judge story is local live mode plus the deep-dive console. This shows the dashboard, backend API responses, WebSocket-style trace, policy block, audit packet hash, PDF audit report, AI proof strip, and E2E proof path.

What the project proves

- AI remediation can be governed by SOPs and policy checks, not just chatbot advice.
- Historical precedent is visible but unsafe precedent is not copied into the plan.
- Human approval is required before mock execution.
- Execution is synthetic and mock-only; no host, cloud, identity, or firewall system is modified.
- RCA, telemetry, audit events, JSON packet, and PDF report are generated for review.

Fastest working path

```
cd "D:\Hackathon - Copy\nexus-resolve"
.\scripts\setup-all.cmd
.\scripts\start-demo.cmd
```

2. Product Concept And Safety Boundary

The project is designed for managed infrastructure teams. It demonstrates how an AI-assisted incident workflow can stay inside enterprise controls: SOP retrieval, historical comparison, policy gate, approval, validation, RCA, and audit export.

Important distinction

Mode	Meaning
Replay Mode	Static event playback from JSONL files. Safe for GitHub Pages. No backend. No secrets. No live OpenAI proof.
Local Live Mode	FastAPI backend on localhost, WebSocket run stream, optional OpenAI Responses API calls from server-side Python only.
Mock Fallback	Deterministic backend response path if OpenAI is unavailable or no key is configured.
E2E Mode	Playwright test mode using isolated ports 8002, 5175, and 5176 so tests do not collide with normal demo ports.

Non-negotiable safety rules

- Never execute real remediation from this demo.
- Never place OPENAI_API_KEY in React, Vite, browser local storage, screenshots, commits, or GitHub Pages.
- Never claim public replay is live OpenAI. Public replay is static by design.
- Never bypass approval. The approval gate is part of the product value.
- Never delete, mutate, or call real enterprise resources from the demo.

3. User Roles And What Each User Does

User	Primary usage
Judge	Open dashboard, inspect one incident, verify proof strip, inspect deep-dive API cards, ask for PDF audit report and E2E proof.
Presenter	Run start-demo.cmd, narrate problem/evidence/governance/approval/RCA/closure, show failure-path scenario if asked.
Operator	Select an incident, review SOP/history/policy, start simulation, approve or reject, close or observe after RCA.
Approver	Review action preview, target resources, safeguards, validation steps, and expected effect before approving.
Developer	Run backend tests, dashboard lint/test/build, E2E suite, maintain scenarios, extend API or UI safely.
QA Reviewer	Run check-all.cmd and run-e2e.cmd, verify replay and local live, confirm no secrets, confirm public replay behavior.
Maintainer/Admin	Protect .env, rotate keys, keep dependencies installable, verify GitHub Pages replay and local live workflow.

Role-specific first action

- Judge: open http://localhost:5173/#/incident/disk-space and http://localhost:5174/apps/deep-dive/#both.
- Presenter: run scripts/start-demo.cmd and keep terminal windows open.
- Operator: use the scenario selector and Start Simulation button.
- Developer: run scripts/check-all.cmd before pushing or presenting.
- QA Reviewer: run scripts/run-e2e.cmd after check-all.cmd.

4. First-Time Setup And Prerequisites

Required tools

Requirement	Expected use
Windows PowerShell or cmd	Run setup, backend, dashboard, and E2E scripts.
Python 3.11+	FastAPI backend, tests, PDF audit report generation.
Node.js and npm	React/Vite dashboard, build, unit tests, Playwright install.
Git	Version control and commit verification.
Network during setup	pip, npm, and Playwright Chromium download.
OpenAI API key	Optional for local live OpenAI mode; not required for replay or fallback.

One-command setup

```
cd "D:\Hackathon - Copy\nexus-resolve"
.\scripts\setup-all.cmd
```

The setup script creates the backend virtual environment if missing, installs backend dependencies including reportlab, installs dashboard dependencies, and installs Playwright Chromium.

Manual backend setup

```
cd services\backend
python -m venv .venv
.\.venv\Scripts\python -m pip install -e .[dev]
.\.venv\Scripts\python -m pytest
```

Manual dashboard setup

```
cd apps\dashboard
npm install
npm run lint
npm run test
npm run build
```

5. Environment Settings And Ports

Copy .env.example to .env only for local live mode. Keep .env local and ignored. Do not commit .env.

Variable	Purpose
OPENAI_API_KEY	Server-side key for optional local live OpenAI Responses API usage. Never expose in frontend.
OPENAI_MODEL	Model name for backend OpenAI calls. Current default in .env.example is gpt-5.5.
APP_MODE	Use live for OpenAI-backed local mode, mock for deterministic fallback/testing.
OPENAI_INPUT_COST_PER_1M_USD	Input token pricing used for estimated API cost telemetry.
OPENAI_OUTPUT_COST_PER_1M_USD	Output token pricing used for estimated API cost telemetry.
HUMAN_HOURLY_RATE_USD	Human comparison rate. Current demo value is 30.
HUMAN_BASELINE_MINUTES_PER_INCIDENT	Manual baseline used for savings comparison. Current demo value is 45.
BACKEND_PORT	Documented backend port. Scripts use 8000 for normal local demo.
FRONTEND_PORT	Documented dashboard port. Scripts use 5173 for normal local demo.

Normal demo ports

Port	Surface
8000	FastAPI backend: http://localhost:8000/api/health
5173	React dashboard: http://localhost:5173/#/incident/disk-space
5174	Deep-dive judge console: http://localhost:5174/apps/deep-dive/#both

E2E isolated ports

Port	Surface
8002	Mock backend started by Playwright.
5175	Dashboard test server started by Playwright.
5176	Deep-dive static server started by Playwright.

6. Running The Project

Recommended judge launcher

```
cd "D:\Hackathon - Copy\nexus-resolve"
.\scripts\start-demo.cmd
```

The launcher reuses existing listeners on ports 8000, 5173, and 5174. If a port is already used by another application, confirm it is the intended NEXUS service before demoing.

Manual launch commands

Surface	Command
Backend	cd services/backend; .venv/Scripts/python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000
Dashboard	cd apps/dashboard; npm run dev -- --host localhost --port 5173
Deep dive	python -m http.server 5174 --bind 127.0.0.1 from repo root

Important URLs

URL	Use
http://localhost:5173/	Operations dashboard home.
http://localhost:5173/#/incident/disk-space	Default judge incident workspace.
http://localhost:5173/#/incident/endpoint-third-party-app-exception	Failure-path rejection scenario.
http://localhost:5174/apps/deep-dive/#both	Judge console showing frontend and backend evidence.
http://localhost:8000/api/health	Backend health check.

7. Dashboard Guide

First screen

The first screen is the active operations alert dashboard. It shows the alert portfolio, critical/high counts, and the executive impact dashboard. Choose an alert card to open the incident workspace.

Incident workspace controls

Control	Meaning
Scenario selector	Switches between the 12 synthetic incidents.
Replay button	Uses static JSONL replay. Safe without backend.
Live button	Uses local backend if online. Optional OpenAI API runs server-side.
Start Simulation	Starts the selected replay or live workflow.
Approve	Unblocks live mock execution after the approval.requested event.
Reject	Ends live workflow safely without mock execution.
Observe	Runs a synthetic observation check before closure.
Close INC	Closes the incident with RCA and audit evidence.
Show Real Block	Fetches /api/policy/demo-block and proves protected-resource blocking.
E2E toggle	Shows the committed Playwright proof panel and command.
PDF Report	Downloads a backend-generated audit PDF for live runs.

What to read on screen

- Ticket Details: team, alert, incident, priority, CI, service, current state, requested outcome.
- Live AI Proof: source, model/backend, run/events, timestamp/fallback, tokens/latency, API cost/human rate.
- Decision Evidence: reason summary, SOP source, history signal, approval decision, selected action, RCA outcome.
- Action Review: human-readable dry-run command preview and validation checklist.
- Policy Checks: target scope, safeguards, dry-run guard, approval, validation, mock-only execution.

8. Judge Deep-Dive Console

The deep-dive console is a static page that helps judges inspect frontend and backend proof side by side. It can show frontend only, backend only, or both screens.

Open it

```
python -m http.server 5174 --bind 127.0.0.1
# Then open:
http://localhost:5174/apps/deep-dive/#both
```

What it proves

- The embedded frontend is the real dashboard route, not a separate mock screen.
- Backend API cards fetch live FastAPI JSON when the backend is running.
- Trace replay is labeled as trace replay when it is not live API data.
- It shows ServiceNow-style mock connector data, policy demo block, run snapshot, audit packet hash, and audit PDF metadata.

How to narrate it

Use the Both Screens view for judging. Start the dashboard simulation, then explain that the backend panel shows the API contract and live evidence path that supports the frontend decisions.

9. Incident Scenario Catalog

Each scenario has a synthetic incident, SOP, history rows, one unsafe precedent, one escalation precedent, before/after metrics, plan template, validation, execution payload, and RCA. The catalog currently contains 12 scenarios.

Scenario	Scenario ID
Windows Infra - Disk utilization high	disk-space
Database - DB connection pool saturation	db-connection-pool
Security / IAM - Suspicious admin role assignment	iam-admin-role
Network - VPN tunnel packet loss	vpn-packet-loss
Linux - Linux server high CPU / load average	linux-high-load
Firewall - Firewall rule blocking application traffic	firewall-rule-block
Backup - Critical server backup failed	backup-job-failed
Service Desk - User account locked repeatedly	servicedesk-account-lockout
AD - Group Policy not applying	ad-gpo-not-applying
Command Centre - Major incident alert storm / duplicate alerts	command-centre-alert-storm
Endpoint Security - Third-party application detected on endpoint	endpoint-third-party-app-exception
Cloud - VM instance unhealthy / failed status check	cloud-vm-unhealthy

Recommended demo paths

- Default happy path: disk-space. It is easy to explain and validates quickly.

- Security governance path: iam-admin-role. Shows preserving evidence and disabling only suspicious grant.

- Failure/rejection path: endpoint-third-party-app-exception. Shows that the system can reject automatic action when an exception exists.

- Command centre path: command-centre-alert-storm. Shows alert correlation and unsafe bulk-close prevention.

10. Local Live OpenAI Mode

Setup

[ ] Copy .env.example to .env.

[ ] Set OPENAI_API_KEY to a real key only in .env.

[ ] Set APP_MODE=live.

[ ] Keep .env ignored and never commit it.

[ ] Run scripts/start-demo.cmd.

Verify live OpenAI

```
.\scripts\verify-live-openai.cmd
```

The dashboard proof strip labels whether evidence came from OpenAI Responses API, deterministic fallback, or replay/static data. The safest judging claim is: local live mode can call OpenAI server-side; public GitHub Pages is replay-only.

If OpenAI is unavailable

The backend emits an openai.fallback event and uses validated deterministic outputs. This is a resilience feature, not a failure of the safety model.

Key handling rules

- Never paste the API key into source code, frontend files, docs, screenshots, or the browser console.

- If a key was exposed in chat, rotate it before submission.

- Use server-side .env only. GitHub Pages cannot safely hold the key.

11. Audit Packet And PDF Report

JSON audit packet

```
GET http://localhost:8000/api/runs/{run_id}/audit-packet
```

The JSON packet includes run id, generated timestamp, SHA-256 audit hash, safety metadata, full run snapshot, events, policy checks, approval record, RCA, and AI telemetry.

PDF audit report

```
GET http://localhost:8000/api/runs/{run_id}/audit-report.pdf
```

The PDF is generated server-side from the same hashed packet. In the dashboard, the Audit Export panel exposes PDF Report and JSON Packet links after a live run starts.

When to use it

- During judging: click PDF Report after approval/RCA if the judge asks for downloadable evidence.
- During QA: verify the endpoint returns application/pdf and starts with %PDF.
- During demos: explain that the PDF is mock/synthetic evidence but follows the enterprise audit packet shape.

What not to claim

- Do not claim the PDF is immutable storage. It is generated from the in-memory run snapshot.
- Do not claim it proves real system remediation. It proves governed synthetic workflow evidence.

12. Impact Dashboard And Cost Telemetry

The impact dashboard shows business value: portfolio time saved, human cost avoided, manual steps avoided, token/latency telemetry, and API cost comparison. The default human baseline is 45 minutes per incident at $30 per hour.

Telemetry fields

Field	Meaning
calls	Number of AI-generation stages captured for the run.
openai_calls	Number of stages that used OpenAI.
fallback_calls	Number of stages that used deterministic fallback.
total_tokens	Token count if returned by the OpenAI SDK response.
total_latency_ms	Total measured latency for AI-generation stages.
estimated_openai_cost_usd	Estimated cost from configured per-1M token prices when usage is available.
estimated_labor_savings_usd	Human baseline cost minus NEXUS MTTR labor cost.

How to explain zeros or pending values

In replay or fallback mode, token usage may be zero or unavailable because no live API token usage was returned. This is expected. In local live mode, usage appears when the SDK response includes usage data.

13. Browser E2E Proof Suite

The project includes a committed Playwright suite. It proves the app path through browser automation instead of relying only on manual smoke tests.

Run it

```
cd "D:\Hackathon - Copy\nexus-resolve"
.\scripts\run-e2e.cmd
```

What it covers

- Dashboard loads on an isolated test port.
- Live workflow starts against a deterministic mock backend on port 8002.
- Approval reaches RCA/waiting closure.
- Audit export panel exposes PDF Report.
- Protected-resource block demo returns the expected blocked policy response.
- Endpoint Security exception replay ends in rejection with no mock execution.
- Deep-dive Both Screens view renders backend evidence.

Why isolated ports matter

The E2E suite uses ports 8002, 5175, and 5176 so it does not collide with the normal demo on 8000, 5173, and 5174. This prevents a different app on 5173 from causing false browser failures.

14. Backend API Reference

Endpoint	Purpose
GET /api/health	Backend health.
GET /api/scenarios	Scenario summaries for backend clients.
GET /api/connectors/servicenow/mock-ticket/{scenario_id}	ServiceNow-style synthetic ticket contract.
POST /api/incidents	Start a live backend run for a scenario.
GET /api/runs/{run_id}	Fetch current run snapshot.
POST /api/runs/{run_id}/approve	Record approval and continue mock execution.
POST /api/runs/{run_id}/reject	Reject the run safely before execution.
POST /api/runs/{run_id}/observe	Run observation window before closure.
POST /api/runs/{run_id}/close	Close incident after RCA.
GET /api/replay/{scenario_id}	Return static replay events for a scenario.
GET /api/runs/{run_id}/audit-packet	Return hashed JSON audit packet.
GET /api/runs/{run_id}/audit-report.pdf	Return downloadable audit PDF.
GET /api/policy/demo-block	Return protected-resource policy block proof.
WS /ws/runs/{run_id}	Stream run events to the dashboard.

15. Demo Script And Judge Flow

3 to 5 minute flow

- Open dashboard and show the impact dashboard.
- Open disk-space incident.
- Start simulation in local live mode if backend and key are ready; otherwise use replay and explain fallback.
- Show SOP, history, unsafe precedent, and policy warning.
- Show generated action review package and Live AI Proof strip.
- Approve the mock remediation.
- Show metrics, RCA, audit trail, PDF/JSON audit export.
- Open deep-dive #both to show live API evidence and backend trace.
- Optionally open endpoint-third-party-app-exception to show rejection proof.

Strong judge sentence

A chatbot suggests. NEXUS-RESOLVE retrieves SOP, compares history, blocks unsafe precedent, requires approval, validates results, emits telemetry, and exports audit evidence.

16. Pre-Demo Dos

[ ] Run scripts/check-all.cmd.

[ ] Run scripts/run-e2e.cmd.

[ ] Confirm .env exists only locally and is not staged.

[ ] Confirm the OpenAI key is valid if showing local live OpenAI mode.

[ ] Start scripts/start-demo.cmd at least 5 minutes before presenting.

[ ] Open dashboard URL and deep-dive URL in browser tabs.

[ ] Check http://localhost:8000/api/health returns ok.

[ ] Use disk-space as the primary scenario.

[ ] Keep endpoint-third-party-app-exception ready as the failure path.

[ ] Prepare one sentence explaining public replay versus local live mode.

Pre-demo don'ts

- Do not start from GitHub Pages if the judge asked to see live OpenAI proof.

- Do not paste or show the API key.

- Do not open unrelated localhost apps on the demo ports.

- Do not skip the approval gate in narration.

17. During-Demo Dos And Don'ts

Dos

- Say synthetic/mock-only early and repeat it when approving remediation.
- Point to SOP beats history as the governance moment.
- Use the Live AI Proof strip to identify OpenAI, fallback, or replay evidence.
- Show the protected-resource block button if judges ask about safety.
- Show PDF Report and JSON Packet if judges ask about auditability.
- Use the E2E toggle or run-e2e output if judges ask how it was tested.

Don'ts

- Do not say the product modifies real infrastructure.
- Do not claim public GitHub Pages proves live OpenAI calls.
- Do not hide fallback; fallback is visible and intentionally labeled.
- Do not describe synthetic connectors as real ServiceNow integration.
- Do not over-demo every scenario. One polished happy path plus one failure path is stronger.

18. Post-Demo Dos

[ ] Stop local demo terminal windows if no longer needed.

[ ] Do not upload .env, logs containing secrets, or screenshots containing keys.

[ ] If the OpenAI key was exposed anywhere, rotate it immediately.

[ ] Run git status and confirm only intended files are changed.

[ ] If publishing, push to main and wait for GitHub Actions/Pages to finish.

[ ] Open the public Pages URL and verify replay mode loads.

[ ] Document any live OpenAI run evidence separately from public replay evidence.

Submission package checklist

- Public GitHub repository link.

- Public GitHub Pages replay link.

- Demo video link or local MP4 if requested.

- README and docs package.

- Local live demo instructions.

- Safety statement: synthetic-only, mock-only, server-side key only.

19. Troubleshooting

Symptom	Fix
The system cannot find the path specified	Confirm you are in D:\Hackathon - Copy\nexus-resolve before running scripts.
Port 8000 already in use	Another backend is running. Reuse it only if it is NEXUS-RESOLVE; otherwise stop that listener or change port manually.
Port 5173 shows wrong app	A different Vite app is on the dashboard port. Stop it, then rerun scripts/start-demo.cmd.
Backend offline in dashboard	Start scripts/dev-backend.cmd or scripts/start-demo.cmd and recheck /api/health.
Live mode falls back	Check OPENAI_API_KEY, APP_MODE=live, network, and API quota. Fallback is safe and visible.
Public page does not show live OpenAI	Expected. GitHub Pages is replay-only. Use local live mode for OpenAI proof.
E2E fails on port conflict	Confirm ports 8002, 5175, and 5176 are free, then rerun scripts/run-e2e.cmd.
PDF link disabled	Start a local live backend run. Replay runs do not have server-side run ids for PDF export.
npm.ps1 blocked	Use npm.cmd or run the provided .cmd scripts.

20. Maintenance And Production Notes

Current hackathon boundary

- Execution is synthetic-only and mock-only.
- Run state is in memory.
- Approval metadata is captured but not backed by real auth/RBAC.
- ServiceNow endpoint is a synthetic connector contract, not a real instance integration.
- GitHub Pages is replay-only.

Production hardening before real deployment

- Add real auth/RBAC and operator identity.
- Use durable append-only audit storage.
- Restrict CORS to trusted origins.
- Add rate limits and budget controls for OpenAI calls.
- Move from mock connectors to properly scoped enterprise connector adapters.
- Add formal policy-as-code if required by enterprise governance.
- Keep destructive operations behind dry-run, approval, change windows, and validation gates.

Roadmap items

- Real OpenAI tool-calling loop for retrieve_sop, retrieve_history, policy_check, validate_result, and write_rca tools.
- Hosted live backend only with server-side secrets, auth, rate limits, and CORS restrictions.
- Durable audit ledger and optional signed PDF export.

21. Command Appendix

Core commands

```
# Setup
.\scripts\setup-all.cmd

# Start local judge demo
.\scripts\start-demo.cmd

# Standard verification
.\scripts\check-all.cmd

# Browser E2E verification
.\scripts\run-e2e.cmd

# Live OpenAI smoke test
.\scripts\verify-live-openai.cmd
```

Backend commands

```
cd services\backend
.\.venv\Scripts\python.exe -m pytest
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Dashboard commands

```
cd apps\dashboard
npm run lint
npm run test
npm run build
npm run dev -- --host localhost --port 5173
npm run e2e
```

Useful API checks

```
Invoke-RestMethod http://127.0.0.1:8000/api/health
Invoke-RestMethod http://127.0.0.1:8000/api/scenarios
Invoke-RestMethod http://127.0.0.1:8000/api/policy/demo-block
```

Final safety reminder

Do not commit .env. Do not expose OPENAI_API_KEY. Do not claim replay is live OpenAI. Do not claim mock execution changed real systems.